ASSIGNMENT 7
COMPUTER NETWORKS
ID: 12341550

PART A:

Algorithms - TCP Cubic, TCP Tahoe (Loss-Based),FAST-TCP (Delay-Based), Compound TCP (Hybrid)

1. **TCP CUBIC [Loss-Based / Estimation]**
   ALGORITHM DETAILS:
   TCP Cubic abandons the standard Additive Increase Multiplicative Decrease (AIMD) linear growth in favor of a cubic polynomial. Its primary innovation is making window growth dependent on the real wall-clock time elapsed since the last congestion event, rendering it completely independent of the path's Round Trip Time (RTT).

   Formula: $W(t) = C * (t - K)^3 + W_{max}$
   - W_{max} = cwnd just before last loss
   - K = time to recover back to W_{max} = $\sqrt[3]{W_{max} * beta / C}$
   - C = 0.4 (scaling constant), beta = 0.7 (reduction factor)
   - t = wall-clock time since last loss event

   Phases:
   1. Slow Start : cwnd doubles each RTT until ssthresh.
   2. Concave Growth : fast recovery toward W_{max} after loss.
   3. Plateau : growth slows near W_{max} (network stabilizes).
   4. Convex Growth : accelerates beyond W_{max}, probing for new bandwidth.

   On loss: cwnd *= 0.7 (gentle 30% cut, NOT reset to 1 MSS)
   RTT-independent: yes window grows based on time, not RTT count.
   TCP-friendly mode: activates on low-BDP networks to not starve Reno flows.

   SUITABLE SCENARIOS:
   - High-bandwidth, high-latency (high-BDP) links intercontinental fiber.
   - Linux servers and data centers (it is the OS default since kernel 2.6.19).
   - Mixed-RTT environments where RTT-fairness matters.

   WHERE IT FAILS:
   - Wireless networks: treats random channel errors as congestion, cuts cwnd needlessly.
   - Latency-sensitive apps: no delay awareness causes buffer bloat.
   - Short-distance LAN: cubic growth provides no advantage over simple Reno.

2. **TCP TAHOE [Loss-Based]**

ALGORITHM DETAILS:

TCP Tahoe (1988) represents the foundational state machine for modern congestion control. It relies on a strict interpretation of packet loss as the sole indicator of network congestion.

Three mechanisms:

1. Slow Start:
   cwnd starts at 1 MSS. Doubles every RTT (exponential) until ssthresh.

2. Congestion Avoidance:
   Once cwnd >= ssthresh, grows linearly: +1 MSS per RTT (AIMD).

3. Fast Retransmit:
   On 3 duplicate ACKs, retransmit lost segment immediately (no timeout wait).

On ANY loss (timeout or 3 DupACKs):
   ssthresh = max(cwnd / 2, 2 * MSS)
   cwnd = 1 MSS <-- always resets to 1, the key weakness

SUITABLE SCENARIOS:

- Teaching and research baselines the canonical example of TCP congestion control.
- Short-RTT, low-loss, reliable wired networks where resets are rare.

WHERE IT FAILS:

- High-BDP networks: recovering from cwnd = 1 takes many RTTs, throughput collapses.
- Wireless / lossy links: frequent resets to 1 MSS devastate performance.
- Cannot distinguish random loss from congestion-caused loss.

3. FAST TCP [Delay-Based]

ALGORITHM DETAILS:

FAST TCP is a proactive, delay-based algorithm. Rather than waiting for a router queue to overflow and drop a packet, it continuously measures queuing delay to estimate the exact number of packets currently buffered in the network.

Formula: cwnd = min(2*cwnd, (baseRTT / RTT) * cwnd + alpha)
- baseRTT = minimum ever-observed RTT (pure propagation, no queuing)
- RTT = current measured round-trip time
- alpha = target number of packets in the network (typically 10–100)

How it works:
- If RTT = baseRTT (no queue): cwnd grows by alpha each update.
- If RTT > baseRTT (queue forming): factor (baseRTT/RTT) < 1, cwnd shrinks.
- Equilibrium: ~alpha packets always sit in the queue; latency stays low.
- No packet loss needed achieves high utilization without drops.

SUITABLE SCENARIOS:

- Dedicated high-speed research/institutional networks (10 Gbps+).
- Latency-sensitive apps: minimizes queue buildup and jitter.
- Stable, controlled data centers where RTT measurement is accurate.

WHERE IT FAILS:
- Mixed deployments: loss-based flows (Cubic, Reno) ignore delay signals and take all bandwidth that FAST TCP gives up. FAST flows get starved.
- Wireless networks: RTT fluctuates due to channel, not congestion causes incorrect cwnd reductions.
- Alpha tuning is sensitive: wrong value leads to under-utilization or queuing.

4. **COMPOUND TCP [Hybrid: Loss + Delay]**

ALGORITHM DETAILS:

Developed by Microsoft, Compound TCP maintains a dual-component architecture to achieve both high utilization on Long Fat Networks (LFNs) and strict fairness with legacy TCP Reno flows. The effective sending window is the sum of a loss-based component and a delay-based component.

Effective window: W = min(cwnd + dwnd, rwnd)

cwnd: standard AIMD — grows +1 MSS/RTT, halves on loss (ensures fairness).
dwnd: delay-based scaling component:
- No congestion (delay low): dwnd += alpha * dwnd^k (superlinear growth)
- Congestion (delay high): dwnd -= zeta * dwnd (proportional cut)
- On loss event: dwnd = 0 (full reset)

Congestion is detected when observed throughput gap exceeds threshold gamma (similar to TCP Vegas delay estimation).

SUITABLE SCENARIOS:
- Windows enterprise and WAN environments (it is the Windows default).
- Mixed networks with both legacy Reno and modern flows cwnd ensures fairness.
- High-BDP intercontinental links where scalable growth is needed.

WHERE IT FAILS:
- High-loss wireless: frequent loss events reset dwnd = 0 and halve cwnd repeatedly.
- RTT noise degrades dwnd component (same sensitivity as FAST TCP).
- Not available natively on Linux creates asymmetry in Windows<->Linux transfers.

5. **COMPARISON AND INFERENCES**

Feature	Cubic	Tahoe	FAST TCP	Compound TCP
Category	Loss+Est.	Loss	Delay	Hybrid
Congestion Signal	Loss	Loss	Delay	Loss+Delay

Loss Response	cwnd*=0.7	cwnd=1MSS	None	cwnd/2, dwnd=0
RTT-Independent	Yes	No	Partial	Partial
High BDP Perf.	Excellent	Very Poor	Excellent	Very Good
Wireless Perf.	Poor	Very Poor	Poor	Moderate
Coexistence	Good	Good	Poor	Good
OS Default	Linux	None	None	Windows

INFERENCES:

1. No single algorithm fits all scenarios.
   Tahoe suits teaching; Cubic suits high-BDP; FAST suits dedicated links;
   Compound suits mixed enterprise. Choice depends on network and application.

2. RTT-independence is essential for high-BDP performance.
   Tahoe's cwnd grows at 1/RTT too slow for long-distance, high-bandwidth paths.
   Cubic (wall-clock based) and FAST TCP (baseRTT/RTT ratio) solve this.
   RTT-independence is a mandatory property for any modern scalable TCP.

3. Loss-based algorithms fail in wireless environments.
   Tahoe and Cubic cannot distinguish congestion loss from random channel errors.
   Tahoe hits cwnd=1 on every wireless drop; Cubic cuts 30% unnecessarily.
   Delay-aware algorithms (FAST, Compound) handle this more gracefully.

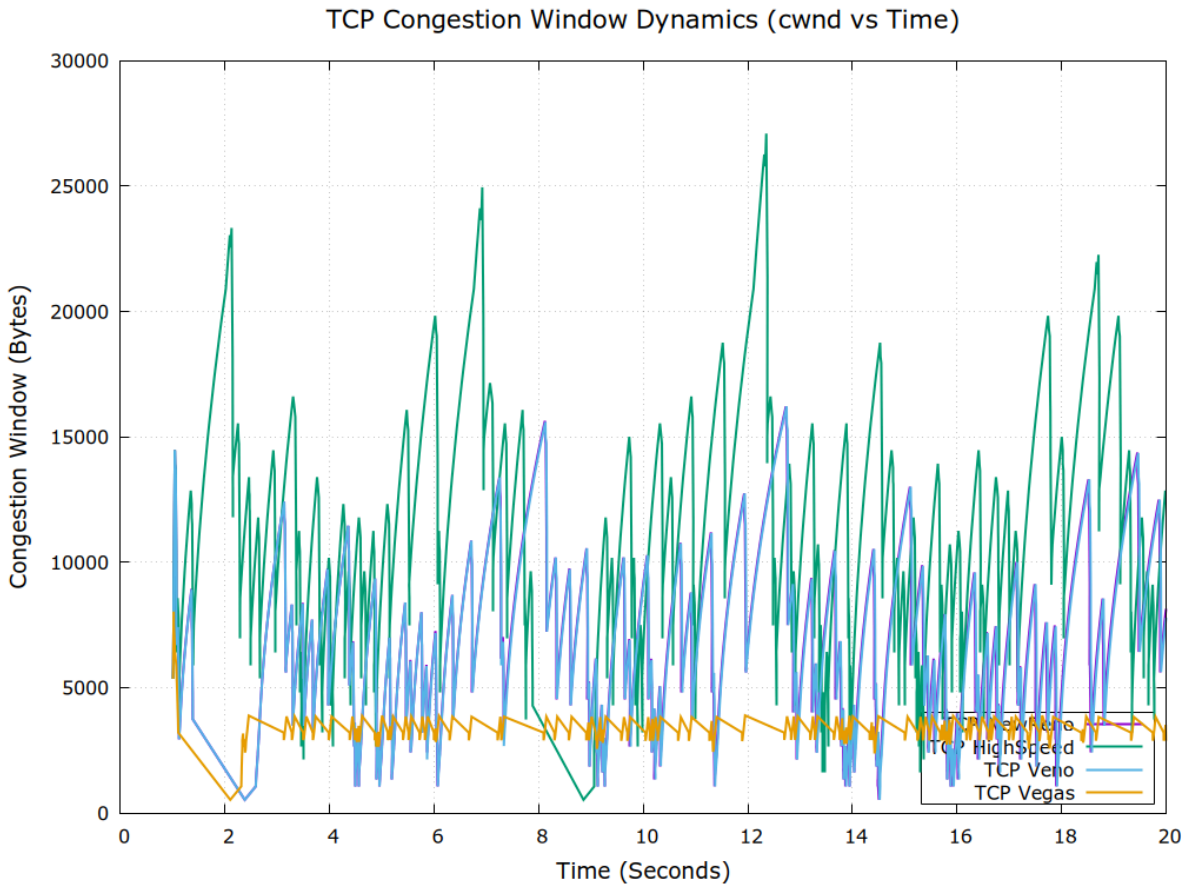
4. Delay-based algorithms are starved in mixed deployments.
   FAST TCP backs off when delay rises, freeing bandwidth that Cubic/Reno grabs.
   This coexistence problem makes pure delay-based TCP impractical in production.
   Compound TCP avoids this by keeping the AIMD cwnd component as a floor.

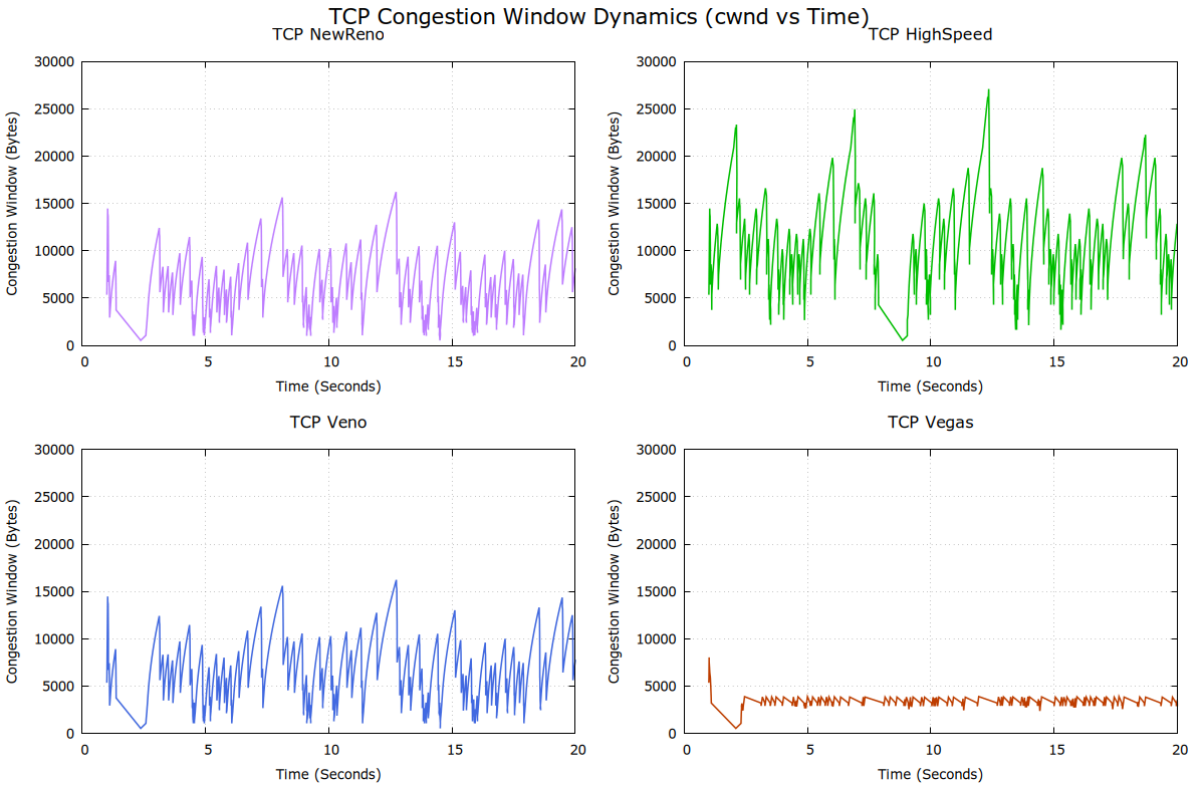
5. Cubic's cubic growth profile is uniquely adaptive.
   Fast concave recovery + stable plateau near W_max + aggressive convex probing
   gives Cubic the right behavior at every phase explaining its Linux dominance.

6. Compound TCP's hybrid design is the most practically robust.
   AIMD cwnd guarantees TCP-friendliness; superlinear dwnd drives throughput;
   dwnd reset on loss provides a safety net missing in FAST TCP.
   Best average performance across diverse real-world conditions.

PART B:

1. **Plot the cwnd vs time graph, and describe what you observed, like slow start and congestion avoidance, in detail.**





1. The Classic "Sawtooth" Pattern (NewReno & Veno):

- **Observation:** Both TCP NewReno and TCP Veno exhibit the classic "sawtooth" shape. We can clearly see the **Slow Start** phase at the very beginning (a steep vertical spike around t = 1.0s) followed by the **Congestion Avoidance** phase (a steady, slower diagonal climb). When the error model drops a packet, the window drops sharply.
- **Inference:** In this specific simulation, Veno's graph is almost identical to NewReno's, peaking around 15,000 bytes. This indicates that Veno's hybrid delay-monitoring mechanics failed to differentiate the random wireless drops (from the RateErrorModel) from actual congestion, causing it to fall back to standard NewReno behavior (halving the window).

2. The Aggressive Probing of HighSpeed:

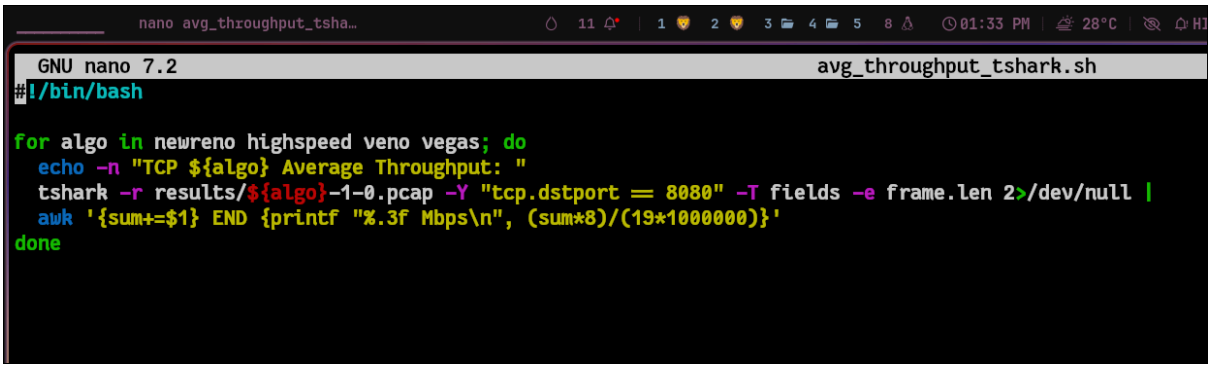
- **Observation:** TCP HighSpeed also shows a sawtooth pattern but reaches significantly higher peaks (up to ~27,000 bytes) compared to NewReno's ~15,000 bytes. The drops occur frequently, but the recovery climbs are incredibly steep.
- **Inference:** This visually proves HighSpeed's design philosophy: it increases its window size much faster than standard TCP during the congestion avoidance phase and doesn't punish itself as severely when a packet is dropped, allowing it to push more data onto the wire overall.

3. The Restraint of Vegas:

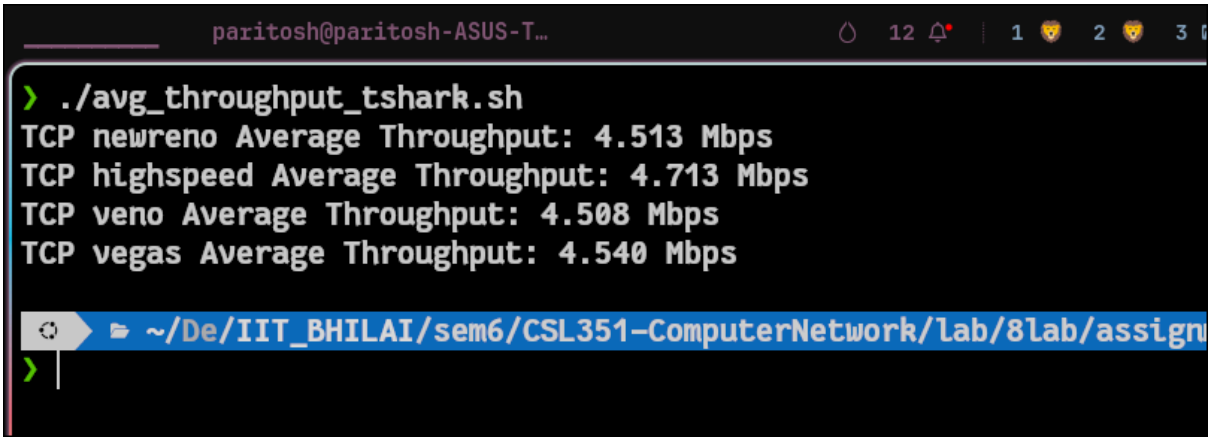
- **Observation:** TCP Vegas is the outlier. After an initial slow start spike, its congestion window completely flatlines, staying rigidly between 3,000 and 4,000 bytes for the rest of the simulation.
- **Inference:** Because Vegas is a delay-based algorithm, it proactively restricts its own window size to prevent the router queue from filling up. It throttles itself so much that it completely avoids the congestion-induced packet drops that cause the sawtooth pattern seen in the other three algorithms.

2. **Find the average throughput for each congestion control algorithm using tshark on the pcap files generated, and state which algorithm performed best.**

   Command:



   Output:



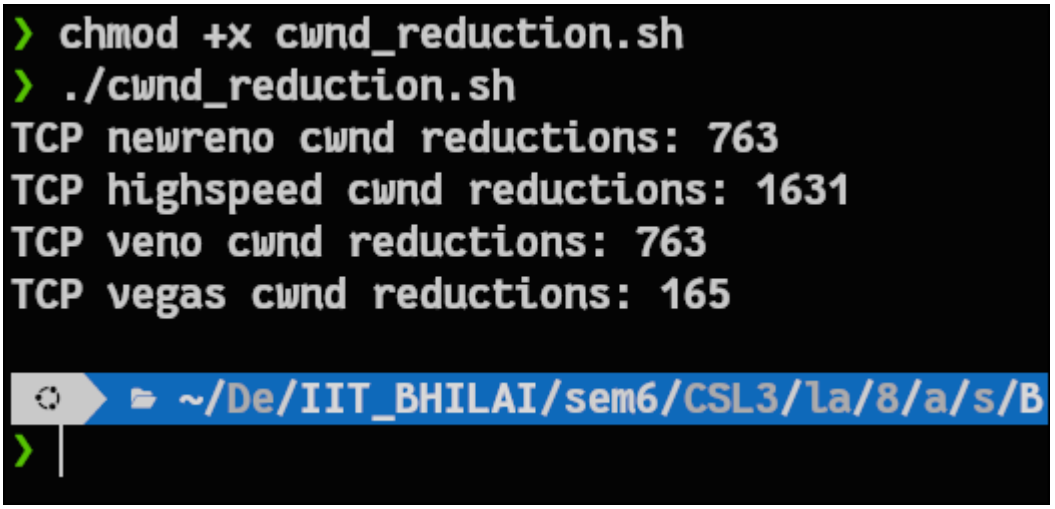
   Throughput is calculated by taking the total number of bytes successfully received by the destination, converting that to bits, and dividing it by the active duration of the connection.

   In Demo_2.cc script, the application starts at 1.0 seconds and stops at 20.0 seconds, meaning the **active transmission time is 19 seconds.** The receiver (sink) is Node 1, so ns-3 will have named your receiver packet capture files ending in -1-0.pcap.

   Based on the simulation data, **TCP HighSpeed** performed the best, achieving the highest average throughput of 4.713 Mbps on the 5Mbps bottleneck link.

HighSpeed's Aggression: TCP HighSpeed is specifically designed to overcome the limitations of standard algorithms on links with higher capacities or minor random losses. When the congestion window is large, HighSpeed modifies its recovery mechanics: it increases the window faster during congestion avoidance and decreases it by a smaller fraction (rather than a strict 50% cut) when a packet is dropped. This allowed it to bounce back rapidly from the random drops caused by the `0.00001` `RateErrorModel` in our topology.
The Vegas Surprise: TCP Vegas managed second place because its delay-based approach prevented it from overflowing the network queues. By keeping the RTT low and steady, it avoided the severe congestion-induced packet drops that trigger massive window reductions.
NewReno and Veno: Both algorithms struggled slightly more with the random error model. NewReno strictly halves its congestion window upon any packet loss, severely bottlenecking its throughput. Veno, while a hybrid algorithm designed to mitigate this, did not show a distinct advantage over NewReno in this specific 20-second, 5Mbps simulation environment.

3. **How many times did the TCP algorithm reduce the cwnd, and why?**



Why these reductions occurred (Inference): The primary trigger for these reductions in this topology is the explicit `RateErrorModel` set to `0.00001`, which introduces random packet loss into the simulation. The algorithms react differently to these drops and the network's queueing dynamics:

a. **TCP HighSpeed (Highest Reductions - 1631):** HighSpeed probes for available bandwidth much more aggressively than standard algorithms. When a packet drops, it reduces its window by a smaller, calculated fraction rather than a strict 50% cut. Because it recovers its window size so quickly and pushes more packets onto the wire, it statistically encounters the random error drops more frequently. It "reduces" more often, but the window stays

larger on average, explaining why it still achieved the highest overall throughput.

b. **TCP NewReno and TCP Veno (763 Reductions):** Both algorithms experienced the exact same number of reductions. NewReno strictly interprets the random packet drops as network congestion and severely halves its window each time. While Veno is designed to differentiate between random loss and congestive loss, in this specific 5Mbps simulation, its delay-estimation mechanics did not prevent it from reacting to the error model's drops in the exact same manner as NewReno.

c. **TCP Vegas (Lowest Reductions - 165):** Vegas behaves proactively rather than reactively. By continuously monitoring the Round Trip Time (RTT), Vegas detected when the router's queue was starting to fill and throttled its own congestion window *before* buffer overflows could cause packet loss. Its 165 reductions were primarily gentle, calculated adjustments based on its internal RTT math to prevent congestion, allowing it to bypass the severe drop-and-halve cycles experienced by the loss-based algorithms.

4. **Check the effect of changing the bandwidth and latency of a point-to-point connection and explain its effect on average throughput.**

Experimental Setup and Results: To test the effects of network parameters, the simulation was run under three distinct scenarios:

a. **Set A (High Delay, Low Bandwidth):** 1 Mbps capacity, 50ms delay
b. **Set B (Baseline):** 5 Mbps capacity, 2ms delay
c. **Set C (Low Delay, High Bandwidth):** 50 Mbps capacity, 1ms delay

Average Throughput Results:

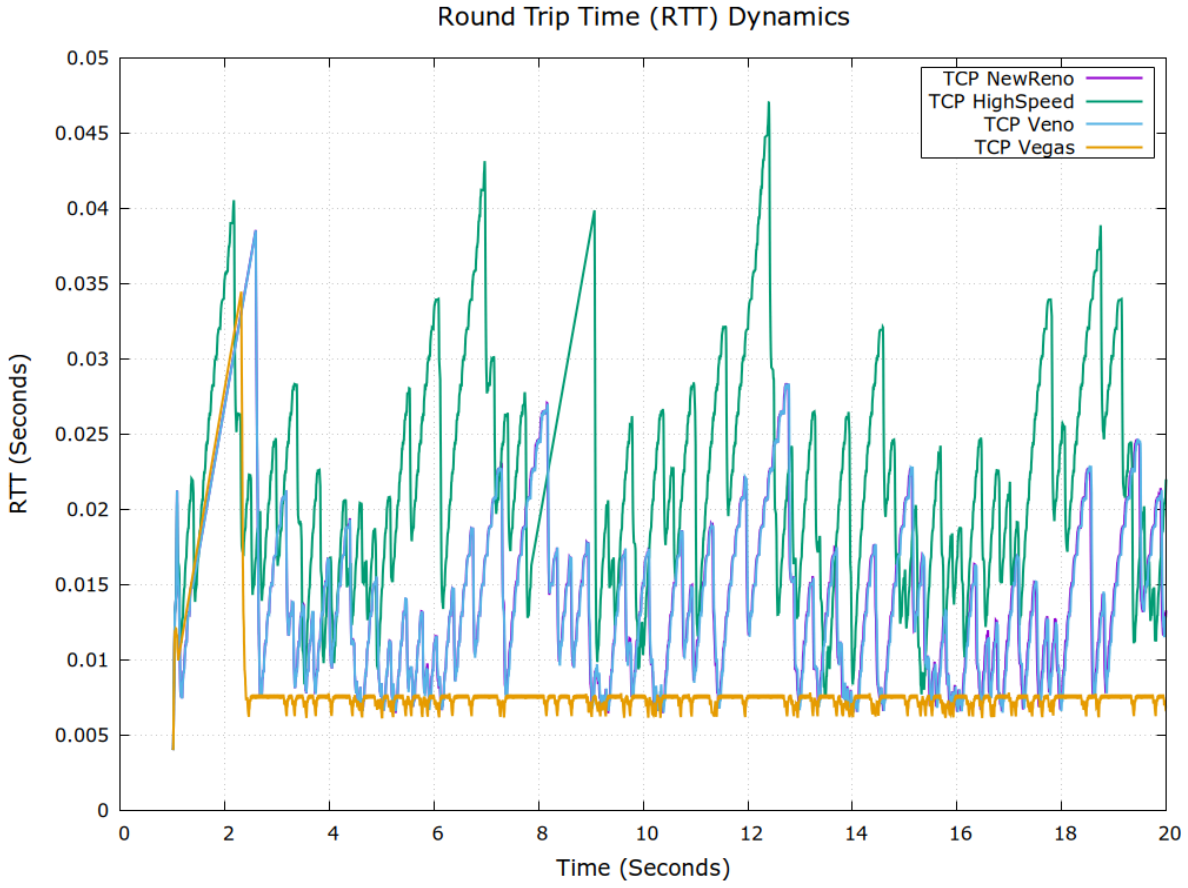
TCP Variant	Set A (1 Mbps, 50ms)	Set B (5 Mbps, 2ms)	Set C (50 Mbps, 1ms)
TCP HighSpeed	**0.773 Mbps**	**4.713 Mbps**	**25.934 Mbps**
TCP Vegas	0.585 Mbps	4.540 Mbps	17.723 Mbps
TCP Veno	0.506 Mbps	4.508 Mbps	18.810 Mbps
TCP NewReno	0.483 Mbps	4.513 Mbps	18.197 Mbps

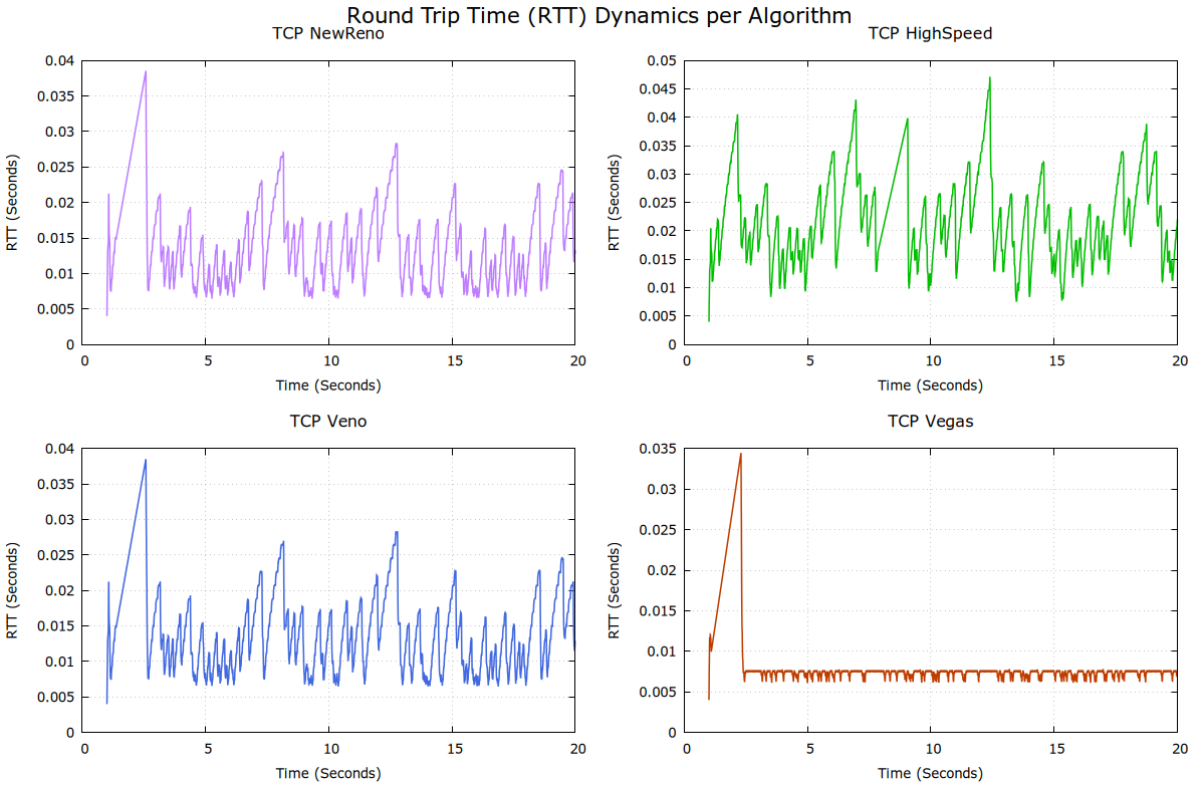
a. **The High Latency Bottleneck (Set A):** When latency was increased to 50ms, the Round Trip Time (RTT) became very high. Because standard TCP only increases its congestion window upon receiving an ACK, this high RTT drastically slowed down the window growth for all algorithms.
HighSpeed still managed the highest throughput because its recovery mechanism is less punishing after a packet drop.
Notably, **Vegas** outperformed standard NewReno and Veno here. On a slow, highly delayed link, Vegas's proactive, delay-based adjustments allowed it to maintain a steady flow without overflowing the small router queues, avoiding the severe timeout and recovery penalties that crippled NewReno.

b. **The High Bandwidth "Fat Pipe" (Set C):** When the link was opened up to 50 Mbps with just 1ms of delay, throughput skyrocketed, but none of the algorithms managed to saturate the full 50 Mbps within the 20-second simulation. This is because the random `RateErrorModel` kept dropping packets, forcing the algorithms to repeatedly cut their windows before they could reach max capacity.
HighSpeed completely dominated this scenario, pulling nearly 8 Mbps ahead of the others. This perfectly demonstrates its core design: it is explicitly engineered to scale its congestion window far more aggressively on high-capacity links and reduce it by a much smaller fraction upon packet loss, allowing it to utilize "fat pipes" much more effectively than standard algorithms.
NewReno, Veno, and Vegas all plateaued around 17-18 Mbps, showing that standard congestion avoidance algorithms are too conservative to quickly fill high-bandwidth links when random packet loss is present.

5. **Explain in short what is the effect of changing the default MTU size.**
Changing the default Maximum Transmission Unit (MTU) size directly impacts the balance between network throughput and reliability. Increasing the MTU improves overall throughput and reduces router processing overhead because more payload data is sent per packet header; however, it makes packet loss more penalizing since a larger chunk of data must be retransmitted, which can stall the TCP congestion window. Conversely, decreasing the MTU reduces maximum throughput due to the higher ratio of header overhead to actual data, but it can make the connection more resilient on highly unreliable links because smaller dropped packets are faster and less costly to retransmit.

6. **Plot the RTT vs time graph and explain your inferences and observations**.


Round Trip Time (RTT) Dynamics
TCP NewReno
TCP HighSpeed
TCP Veno
TCP Vegas
RTT (Seconds)
Time (Seconds)
0
0.005
0.01
0.015
0.02
0.025
0.03
0.035
0.04
0.045
0.05
0
2
4
6
8
10
12
14
16
18
20


Round Trip Time (RTT) Dynamics per Algorithm
TCP NewReno
TCP HighSpeed
TCP Veno
TCP Vegas
RTT (Seconds)
Time (Seconds)
0
5
10
15
20

1. The Link Between cwnd and Queueing Delay:

- **Observation:** If you compare the NewReno, Veno, and HighSpeed RTT graphs to their cwnd graphs, they mirror each other perfectly. As the congestion window grows, the RTT climbs steadily. When the window drops, the RTT plummets.
- **Inference:** This demonstrates queueing delay. The baseline propagation delay is fixed, but as the algorithms pump more packets into the network, those packets get stuck waiting in the router's buffer. The larger the window, the longer the queue, and the higher the RTT.

2. HighSpeed Causes High Latency:

- **Observation:** HighSpeed pushes the RTT to the highest maximums of the group, peaking near 0.047 seconds.
- **Inference:** Because HighSpeed maintains the largest congestion window (as seen in Q1), it naturally fills the router buffers the most, causing the most severe queueing delay for its packets.

3. Vegas Eliminates Queueing Delay:

- **Observation:** After the first two seconds, the RTT for TCP Vegas drops to around 0.007 seconds and stays completely flat, with only microscopic fluctuations.
- **Inference:** This proves the core mechanic of Vegas. By artificially keeping its congestion window small, it ensures the router queue is almost always empty. Packets flow straight through without waiting, resulting in the lowest, most stable latency possible, at the direct cost of throughput.